

Installing the Stray-Light Analysis Toolchain

A Setup Guide for Optical Engineers

30 July 2026

Abstract

This guide installs the software required to run a closed-loop stray-light analysis: sequential design in Ansys Zemax OpticStudio, non-sequential simulation in Ansys Speos, and scripted mechanical geometry in CadQuery. It assumes you are an optical engineer, not a software developer. No programming is required to follow it, and every step ends with a check that tells you whether that step worked before you move on.

Budget about ninety minutes, most of which is unattended installer time. Sections 2 and 3 require Ansys licenses; Sections 4 onward do not.

All paths and version numbers in this guide were verified on a reference machine running Windows 11, last on 8 August 2026. Where your own installation differs, you do not edit the scripts: the four machine-specific paths live in one configuration file (Step 4), and everything else is derived. The sequence does not change.

Contents

1	Before you start	3
1.1	What you are installing, and why	3
1.2	Requirements	3
2	Step 1 — Ansys Zemax OpticStudio	3
3	Step 2 — Ansys Speos	4
3.1	The headless launcher	4
3.2	The bundled Python interpreter	4
3.3	The results viewer component	5
4	Step 3 — The CadQuery environment	5
5	Step 4 — The pipeline scripts	6
5.1	Place it	6
5.2	Configure it — four values, one file	6
5.3	Verify it before running anything	7
5.4	The one dependency that is not on PyPI	7
6	Step 5 — Run the pipeline	7
6.1	First run — prove the installation before you supply anything	7
6.2	The bundled sample set	8
6.3	Stage your own design	8
6.4	Then two commands	9
6.5	Bounding the wall-model uncertainty	9
7	The license seat: the one thing that will bite you	10
7.1	What happens	10
7.2	Two things that quietly compete for the seat	11
7.3	The <i>other</i> entitlement: Ansys OPTIS HPC	11
8	Optional — building the reports	12
9	Verifying the whole installation	12
10	Troubleshooting	13

1 Before you start

1.1 What you are installing, and why

Four pieces of software cooperate. It is worth understanding the division of labour before installing anything, because most setup errors come from expecting one component to do another's job.

Table 1: The four components and their roles.

Component	Role	Why it is needed
Ansys Zemax OpticStudio	Sequential design and baseline metrics	Holds the prescription; exports it for simulation
Ansys Speos	Non-sequential ray tracing	Traces rays where physics sends them, including into the barrel wall
Ansys CPython 3.10	Reads simulation results	Ships with Speos; the only interpreter that can load the result-file bindings
CadQuery (separate Python)	Builds mechanical geometry	Generates the barrel and baffles as STEP files from the prescription

Note that there are *two* Python interpreters, and they are not interchangeable. This is the single most common source of confusion, so it is worth stating plainly:

- The **Ansys CPython** that ships inside Speos reads simulation result files. A normal Python installation cannot do this — the bindings will not load.
- A **separate CadQuery Python** builds geometry. Ansys CPython cannot do this, because CadQuery is not installed into it.

You will not have to remember which is which: each command in this guide gives the full path to the interpreter it needs.

1.2 Requirements

- Windows 10 or 11, 64-bit.
- Licenses for OpticStudio and Speos. **One OpticStudio seat is sufficient**, and in fact the workflow assumes exactly one (Section 7).
- An **Ansys OPTIS HPC** entitlement, which Speos checks out separately whenever it solves. It is easy to overlook because it is not the OpticStudio seat and fails differently — quietly, and with plausible results (Section 7.3).
- Approximately 40 GB of free disk space.
- Administrator rights for the two Ansys installers only.
- An internet connection during installation **and while running** — the HPC checkout is a web call, so an expired Ansys sign-in stops simulations even on a fully installed machine.

2 Step 1 — Ansys Zemax OpticStudio

Install OpticStudio 2026 R1 using the standard Ansys installer and accept the default location. On the reference machine this is:

```
C:\Program Files\Ansys Zemax OpticStudio 2026 R1.00
```

No special options are required. The programming interface this workflow uses (the ZOS-API) is installed automatically as part of the normal product; there is no separate component to select.

Verify

Open PowerShell and confirm the interface libraries are present:

```
dir "C:\Program Files\Ansys Zemax OpticStudio 2026 R1.00\ZOSAPI*.dll"
```

You should see three files: ZOSAPI.dll, ZOSAPI_Interfaces.dll and ZOSAPI_NetHelper.dll. If any are missing, the installation is incomplete — repair it before continuing.

3 Step 2 — Ansys Speos

Install Speos 2026 R1 (Ansys release v261). Accept the defaults. Two things are installed that you will use directly, so it is worth confirming both now rather than discovering a gap later.

If you are not on v261

Substitute your own release number for v261 everywhere below, including in the two verification commands. Ansys releases install side by side rather than replacing one another — the reference machine carries v241, v242, v251 and v261 at once, each with its own Optical Products tree. On a different release the paths printed here simply will not exist, and the Test-Path checks at the end of this step will return **False** for that reason alone rather than because anything is wrong with your installation.

3.1 The headless launcher

Speos can be driven without opening its user interface. The program that does this is:

```
C:\Program Files\ANSYS Inc\v261\Optical Products\Speos\bin\AnsysSpeosLauncher.exe
```

It runs a script and exits. You will not normally type this command yourself — the pipeline issues it — but recognizing it in a log file is useful:

```
AnsysSpeosLauncher.exe /RunScript=<script.py> /Headless=True  
                        /Splash=False /Welcome=False /ExitAfterScript=True
```

3.2 The bundled Python interpreter

Speos installs its own CPython 3.10, which is the interpreter that can read simulation result files:

```
C:\Program Files\ANSYS Inc\v261\commonfiles\CPython\3_10\winx64\Release\python\python.exe
```

Do not substitute a normal Python here

The result-file bindings are compiled against this specific interpreter. Installing Python 3.10 from python.org and pointing the scripts at it will fail at import, and the error message will not obviously say why. Use the path above.

3.3 The results viewer component

Simulation irradiance maps are read through a component called **XMPViewer**, registered automatically by the Speos installer.

Verify

Confirm all three parts of the Speos installation:

```
Test-Path "C:\Program Files\ANSYS Inc\v261\Optical
↪ Products\Speos\bin\AnsysSpeosLauncher.exe"
Test-Path "C:\Program Files\ANSYS
↪ Inc\v261\commonfiles\CPython\3_10\winx64\Release\python\python.exe"
Test-Path "HKLM:\SOFTWARE\Classes\XMPViewer.Application"
```

All three must print **True**. If the third prints **False**, repair the Speos installation — the component registration did not complete.

4 Step 3 — The CadQuery environment

CadQuery builds the barrel, baffles and housing as solid geometry driven by the lens prescription. It needs its own Python installation, kept separate from everything above. Install Python 3.11 or later from python.org if you do not already have one, then create the environment. This guide uses `C:\flv` to match the reference machine; any path will do, provided you use it consistently.

```
python -m venv C:\flv
C:\flv\Scripts\python.exe -m pip install --upgrade pip
C:\flv\Scripts\python.exe -m pip install cadquery numpy scipy ansys-speos-core
```

Once you have the pipeline (Step 4) you can install exactly what it needs instead, which is the same list kept beside the code:

```
C:\flv\Scripts\python.exe -m pip install -r requirements.txt
```

ansys-speos-core (PySpeos) reads the **.lpf ray files** the stray-angle measurement depends on. It talks to `SpeosRPC_Server.exe`, which already ships inside the Ansys install (Section 3) — there is nothing further to obtain, but note that the server checks out one Speos seat for as long as it is running, and the pipeline therefore starts one per analysis pass rather than one per file.

The reference machine runs the versions in Table 2. Later versions are expected to work; these are simply what has been tested.

Table 2: Verified package versions on the reference machine.

Package	Version
cadquery	2.8.0
cadquery-ocp	7.9.3.1.1
numpy	2.4.6
scipy	1.18.0
ansys-speos-core	0.9.1

Verify

Confirm CadQuery loads and can build a solid:

```
C:\flv\Scripts\python.exe -c "import cadquery as cq;  
↪ print(cq.Workplane().box(1,1,1).val().Volume())"
```

This should print 1.0. A long error mentioning **OCP** means the geometry kernel did not install correctly; reinstall **cadquery-ocp** before continuing.

5 Step 4 — The pipeline scripts

5.1 Place it

The pipeline is a single directory of about fifty source files. Unpack it wherever you intend to keep it — any path will do, and it does not have to match this guide. Everything inside locates itself relative to that directory, so there are no paths to edit in the code.

Table 3: Pipeline layout.

Directory or file	Contents
lib\ survey\ straylight.toml.example	The runner, job manifests and safety checks The generic, prescription-driven pipeline The four paths you must supply
LICENSE, NOTICE.md	Terms, and third-party attribution

5.2 Configure it — four values, one file

Four things cannot be derived, because they are where *your* Ansys and Python installations happen to live. They go in one file. Copy the example beside it:

```
copy straylight.toml.example straylight.toml
```

Then open **straylight.toml** and set the four paths — the example file explains what each one is and how to recognise the right folder:

```
[paths]  
ansys_root      = "C:/Program Files/ANSYS Inc/v261"  
opticstudio_root = "C:/Program Files/Ansys Zemax OpticStudio 2026 R1.00"  
python_exe      = "C:/flv/Scripts/python.exe"  
zemax_data      = "C:/Users/<you>/Documents/Zemax"
```

The first three are the installations from Steps 1–3. The fourth is your OpticStudio user-data folder, the one holding `Samples\`; leave it as `"` if you are supplying your own lens files. Forward slashes are fine on Windows and avoid escaping problems.

5.3 Verify it before running anything

```
python lib\settings.py --check
```

This prints every path the pipeline will use — both the four you supplied and the three it derives from them — and says whether each one exists:

```
ok ansys_root      C:\Program Files\ANSYS Inc\v261
ok python_exe     C:\flv\Scripts\python.exe
ok opticstudio_root C:\Program Files\Ansys Zemax ...
ok -> speos_launcher ...\.Speos\bin\AnsysSpeosLauncher.exe
ok -> ansys_cpython ...\.CPython\3_10\...\python.exe

CONFIG OK
```

Do not skip this

It is the only step between installing and running, and it takes a second. A wrong path here does not announce itself later: a mistyped Speos location launches nothing and leaves an empty log, which looks exactly like a simulation that found no stray light. The check turns that into a named error against a named key. If a path is wrong it reports *every* problem at once, distinguishing “does not exist” from “exists, but is not that kind of folder.”

If you later upgrade Ansys — v261 to v262, or a new OpticStudio release — this one file is the only thing to change.

5.4 The one dependency that is not on PyPI

Surface-scatter data is converted by a package called `ansys_optical_automation`, which is **not available through pip**. It is not included here, because nothing in the workflow imports it.

You need it only if you intend to author your own scatter (BSDF) data. If you do, obtain it from its public source repository rather than searching PyPI, where you will not find it. The main workflow runs without it, using the scatter file shipped alongside the scripts.

6 Step 5 — Run the pipeline

6.1 First run — prove the installation before you supply anything

Do this before staging a design of your own:

```
python lib\first-run.py
```

It generates a known-good example lens, builds its job manifest, and runs `preflight` on it — about ten seconds of OpticStudio. A verdict of `G0` means your installation works end to end. The point is not convenience. Until this existed, your first act was to supply a design *before* you had any evidence the installation worked, so a failure could be your configuration, your Ansys install, your lens, or the pipeline, with no way to tell which. Because this example is

known to pass, a failure here is your installation — and the tool ranks the four candidate causes for you, licence seat first, because that is the usual one.

Add `--full` to take the same example through all eight stages and print a real stray-light number. That takes about nine minutes on the reference machine and holds the HPC entitlement throughout (§7).

Known answer

The `--full` run is also a correctness check, not just a smoke test. On the reference machine it returns $-95.6\% \pm 1.3\%$ stray-light reduction with imaging throughput unchanged at -0.4% . If your figure is close to -95% , your installation is not merely running — it is producing the right answer.

The example is **generated, not shipped**: no `.zmx` files are redistributed anywhere in this package, because the designs the workflow was developed against are stock OpticStudio samples. `first-run-lens.ps1` holds the prescription as numbers and *your* OpticStudio writes the file.

6.2 The bundled sample set

Four designs ship the same way, spanning deliberately different archetypes:

```
python lib\make-samples.py
python lib\run-fleet.py
```

They exist so you can reproduce the range of answers this workflow produces, rather than take it on trust. Measured on the reference machine:

design	f/#	field	stray reduction
example-triplet	f/5	$\pm 14^\circ$	-95.6% decisive
wfov-30	f/4	$\pm 30^\circ$	-18.3% decisive
fast-f2p5	f/2.5	$\pm 10^\circ$	-2.7% barely significant
longbore-f8	f/8	$\pm 4^\circ$	-0.7% no change

Table 4: The four bundled designs. All carry the same $\pm 1.3\%$ Monte-Carlo uncertainty.

That last row is the most useful one. A workflow that always reports a large improvement is a brochure; this one will tell you when a redesign is not worth building, and that verdict is worth more than a flattering number because it is the one that saves you the part.

6.3 Stage your own design

The pipeline works on lens prescriptions you supply; none ship with it. Put each design in its own folder under `survey\systems`, where **the folder name and the file name must match** — that name becomes the job’s name everywhere afterwards:

```
survey\systems\<name>\<name>.zmx
```

So a design you call `my-lens` goes at `survey\systems\my-lens\my-lens.zmx`. The name is yours to choose; nothing in the code needs editing to add one.

One naming rule

Every Speos artefact a run produces is tagged with the *first four characters* of the name. Two designs starting with the same four characters would overwrite each other's results, so `make-manifests.py` refuses and tells you which two to rename. `lens-a` and `lens-b` collide; `cooke-a` and `tessar-b` do not.

6.4 Then two commands

Generate a job manifest for every staged design:

```
python lib\make-manifests.py
```

It reports one line per design. If it says `no systems found`, nothing is staged yet — see above. Then run them:

```
python lib\run-fleet.py
```

The runner works through eight stages in order:

`preflight` → `layout` → `odx` → `mech` → `s0` → `sim_optics` → `sim_base` → `sim_redesign`

Three properties are worth knowing before your first run:

- **It is resumable.** If a run is interrupted, starting it again picks up from the last completed stage rather than repeating work.
- **It refuses rather than guesses.** If your design fails a suitability check, the run halts at `preflight` in about ten seconds and tells you which check failed. This is the intended behavior, not an error.
- **It is safe to leave unattended.** A full pass takes roughly forty minutes per design, almost all of it simulation.

Triage a batch cheaply

Once your installation is proven (§6), run only the first stage across your own designs before committing hours of simulation:

```
python lib\run-fleet.py --only preflight
```

This tells you which designs the workflow can actually analyze, in about ten seconds each, rather than finding out forty minutes into a run.

6.5 Bounding the wall-model uncertainty

The wall scattering model that ships here, `black-anodize-plausible.anisotropicbsdf`, is **synthetic** — a physically reasonable stand-in, not measured data — and it is one of the largest uncertainties in the whole workflow. A stray-light number quoted without reference to it is quoting an unstated assumption.

So bound it rather than trusting it. Two more copies of the same physical model ship alongside, scaled to half and double the reflectance:

```
python run-bsdf-band.py --samples
```

This re-runs the four sample designs across a *fourfold* span in wall reflectance and reports how far the headline reduction moves.

design	low	shipped	high	spread
example-triplet	−96.5%	−95.6%	−94.3%	2.2 pp
wfov-30	−12.7%	−18.3%	−25.5%	12.7 pp
fast-f2p5	−2.4%	−2.7%	−2.7%	0.3 pp
longbore-f8	−1.4%	−0.7%	−0.4%	0.9 pp

Table 5: Wall-BSDF band. The reduction is a ratio of two runs sharing the same walls, so it is far more stable than either absolute flux.

The reduction survives the swing: 0.3 to 12.7 percentage points of movement on benefits spanning −0.7% to −95.6%, and for three of the four designs the *conclusion* does not change at all. **longbore-f8** is not statistically significant at any wall model, so “this design does not benefit from vanes” holds across the whole span. **wfov-30** is the case that needs care: the benefit roughly doubles across the band, so quote it as a range, never as a bare number.

When the band decides the answer

A band that moves the *magnitude* is a caveat you state and move on from. A band that spans “clear benefit” to “no benefit” means the simulation does not answer the question for that design without measured BSDF data — and that case is real. If your housing’s surface finish matters to your conclusion, measured BSDF for your actual coating at 60–75° angles of incidence is the highest-value input you can add.

7 The license seat: the one thing that will bite you

Almost every difficult problem reported with this toolchain has the same root cause, and it is worth one page of its own.

7.1 What happens

OpticStudio has a **single license seat**. Only one process may hold it at a time. If a run crashes or is killed, the seat is *leaked* — still recorded as held, by a process that is no longer doing anything.

The symptom is not an error message. It is one of two things:

1. Later runs **block forever**, with no error and, confusingly, *no OpticStudio window visible*. That absence is the diagnostic.
2. Runs **slow down dramatically** rather than stopping — from roughly 80 seconds per design to 5 minutes.

How to recognize it

Look for a live OpticStudio process that is using *no CPU*. That combination — running, but idle — is the signature of a leaked seat. Ending that process restores full speed immediately.

The runner reaps stale processes automatically before each stage that needs the seat, so you should rarely have to do this by hand. If you do:

```
Get-Process OpticStudio,ANSYSHelpViewer -ErrorAction SilentlyContinue |  
Stop-Process -Force
```

7.2 Two things that quietly compete for the seat

The Ansys help viewer.

ANSYSHelpViewer has been observed holding the seat three times in a single working session. It is easy to overlook because opening documentation does not feel like starting a simulation.

An OpticStudio automation server.

If you run any tool that keeps an OpticStudio session open in the background, it holds the same single seat, and every scripted run will block indefinitely. Close it before running the pipeline.

7.3 The *other* entitlement: Ansys OPTIS HPC

The seat above is OpticStudio's. Speos checks out a **second and unrelated** entitlement when it solves — **Ansys OPTIS HPC**, requested as a number of parallel tasks (ten on the reference machine). Everything above can be perfectly healthy while this fails.

It matters far more than an ordinary error, for one reason:

Speos does not treat a licensing failure as a run failure

The simulation reports itself as **computed**, records the problem only in its **StatusInfo**, and carries on. Sometimes no report is written. **Sometimes the report files are written anyway** — so “the output files are all there” is *not* evidence that a stage succeeded, and a resumed run will mark that stage ok.

Both failures print the same first line — Web Licensing checkout call to Direct LaaS REST API for n tasks of Ansys OPTIS HPC failed — and then diverge:

Status 404 — “No valid products found or provisioned servers available”.

Either your account carries no HPC entitlement at all, or one is entitled but already in use. **A second live Speos instance is enough to cause it.** Run one at a time — and that is a hard constraint, not a matter of probability: measured on the reference machine, a *single* solve checks out **the entire HPC pool** (optishpc 10/10) for its duration, so a concurrent solve has nothing left to take, whatever the pool size.

Status 499 — “timed out waiting for ANSYSID authorization response: Unable to retrieve updated token”.

Nothing to do with concurrency: your Ansys sign-in has expired. Re-authenticate and re-run the stage.

What the pipeline does about it, and what you should check

Because a half-completed sweep produces *plausible* numbers rather than an error, the pipeline refuses to infer success from the presence of files. Every simulation's **StatusInfo** is tested, and each result file ends with a machine-readable verdict:

```
SWEEP OK -- 1 sims, 0 failed  
SWEEP INCOMPLETE -- 2 failed
```

The one line to look at

Read the last line of the result file. It is the only statement in the output that distinguishes “this measurement is complete” from “this measurement is missing half its data”. A partial sweep is refused rather than averaged.

This is not hypothetical. Before that check existed, a sweep that lost one of its two samples to a 404 silently degraded to a single measurement — which also removed the repeat-pair noise estimate, so the run reported an error of zero. The result was a confident figure built from half its data. One historical measurement was affected and had to be withdrawn and re-run.

8 Optional — building the reports

The written reports in this project are typeset in L^AT_EX. If you want to rebuild them, the simplest option is Tectonic: a single executable that downloads the typesetting packages it needs on first use, with nothing else to install and nothing added to your system. Download the current Windows release from the project’s releases page on GitHub:

```
Repository:  tectonic-typesetting/tectonic
Asset:       tectonic-<version>-x86_64-pc-windows-msvc.zip
```

Extract it anywhere — no installer is involved — and run:

```
tectonic.exe -X compile report.tex
```

The first run takes a few minutes while it fetches packages; later runs take seconds. This document was produced that way.

A harmless message

On Windows, Tectonic prints `Fontconfig error: Cannot load default config file.` This is noise and can be ignored — the document still builds correctly.

9 Verifying the whole installation

Run these four checks in order. Each isolates one component, so the first one that fails tells you exactly which step to revisit.

1. **The configuration is right** — `python lib\settings.py --check` prints `CONFIG OK` (Section 5). This single command replaces four separate file-existence checks: it verifies the OpticStudio libraries, the Speos launcher and the bundled Ansys CPython in one pass, and names whichever one is wrong.
2. **Results viewer registered** — `XMPViewer.Application` is in the registry (Section 3). Not a path, so the check above cannot see it.
3. **CadQuery builds a solid** — the box test prints 1.0 (Section 4).
4. **The pipeline gate runs** — `run-fleet.py --only preflight` returns a verdict (Section 6).

If all four pass, the installation is complete. Check four is the meaningful one: it exercises the license seat, the design interface and the pipeline’s own logic together, which is more than the preceding three do individually.

10 Troubleshooting

Table 6: Symptoms and their usual causes.

Symptom	Usual cause and remedy
A run hangs with no error and no OpticStudio window	Leaked license seat. End the idle OpticStudio process (Section 7).
Runs became roughly four times slower	Also a leaked seat — it throttles as often as it hangs. Same remedy.
An import error mentioning the result-file bindings	The wrong Python. Use the Ansys CPython inside Speos, not a python.org installation (Section 3).
A long error mentioning OCP	The CadQuery geometry kernel did not install. Reinstall <code>cadquery-ocp</code> (Section 4).
<code>pip</code> cannot find <code>ansys_optical_automation</code>	Correct — it is not published on PyPI, and it is not included here. Obtain it from its source repository, and only if you intend to author scatter data (Section 5).
Anything reports a file or folder that does not exist	Check the configuration before anything else: <code>python lib\settings.py --check</code> (Section 5).
A result file ends <code>SWEEP INCOMPLETE</code>	A simulation failed while the rest succeeded — usually an HPC license checkout. The measurement is refused rather than averaged. Re-run the stage (Section 7.3).
Ansys OPTIS HPC failed, status 404	No HPC entitlement available — most often a second Speos instance is already using it. Run one at a time (Section 7.3).
Ansys OPTIS HPC failed, status 499	Your Ansys sign-in expired. Re-authenticate, then re-run (Section 7.3).
The run halts at <code>preflight</code>	Not a fault. Your design failed a suitability check; the message names which one (Section 6).
Export fails citing aperture parameters	The design has differing front and back aperture settings on a surface, which the exporter cannot represent. Reconcile them in OpticStudio.

A closing note on error messages. Several failures in this toolchain return a plausible wrong value rather than raising an error. The pipeline contains checks for the known cases, which is why a halt should be read as the system working rather than failing. If a result looks surprising but not impossible, treat that as a reason to investigate rather than to proceed.